6. Training CNNs and CNN Architectures

> **CNN 구성:** CNN 구성 층 → 활성화 함수 → CNN 구조 → 가중치 초기화
>
> **CNN 학습:** 데이터 전처리 → 데이터 증강 → 전이학습 → 하이퍼파라미터 선택

How to Build

1. Layers in CNNs

💡 **구성 요소**

- **Convolution**: filter → 공간 특징 추출.
- **Pooling**: 공간 크기 축소.
- **Fully-connected**: 입력 특징을 연결해 출력 계산.
- **Normalization**: 정규화 ⇒ learnable parameter: 크기 · 위치 조정.
- **Dropout**: training 중 일부 activation을 random하게 0으로 설정.
- **Activation function**: nonlinearity 추가.

Normalization

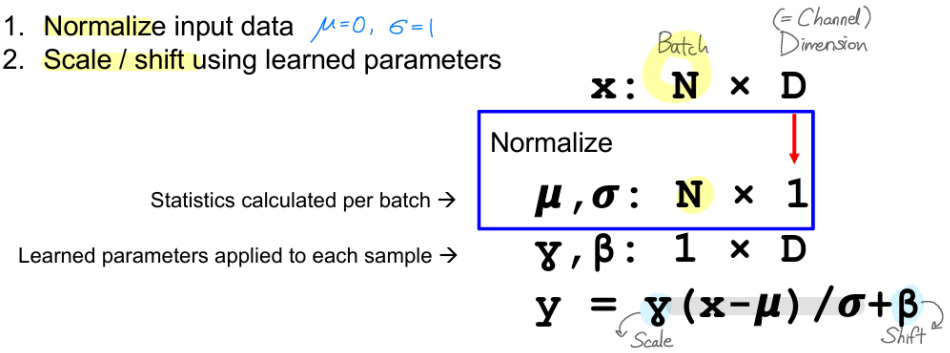
거시적 관점: learnable parameter 학습 → 입력 data의 scale · shift

- normalization 통계 → sample별로 계산. scale·shift parameter → sample 사이에서 공유.
- x (원본) ⇒ $\hat{x}$ (norm) ⇒ $\gamma\hat{x} + \beta$ (Scaling & Shift)
  1. input data의 normalization
  2. γ: 크기 조정(scale), β: 위치 이동(shift).

     $y_{i,j} = \gamma_j \frac{x_{i,j} - \mu_i}{\sigma_i} + \beta_j$

(예시) LayerNorm

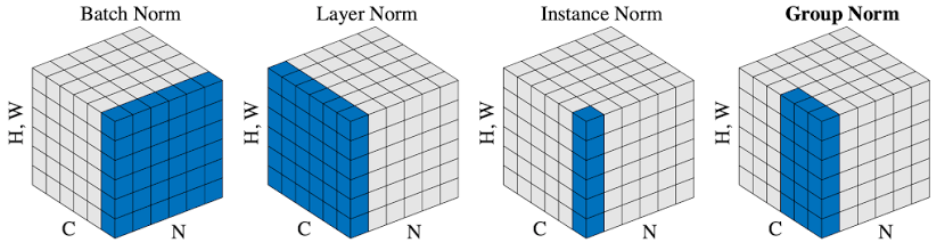
1. Normalize input data $\mu=0,\ \sigma=1$
2. Scale / shift using learned parameters



Normalization 종류

CNN 입력 **(N × C × H × W)** ⇒ normalization 축 상이.

(N: batch 수. C: Channel. R: resolution(H x W))



방식	통계(μ, σ) 축	장점	단점	주요 응용
BatchNorm	C (mini batch)	• 학습 속도 향상 • 높은 학습률 사용 가능	• 작은 batch 불안정 • RNN 적용 어려움	CNN
LayerNorm	N	• batch-free • RNN 적용 가능	• CNN 성능 저하	RNN
InstanceNorm	(data 1개) C	• style 전이 효과적 • batch size-free	• Channel간 정보 무시	Style 전이 Image 생성
GroupNorm	(data 1개) C Group	• 작은 batch에서도 안정적 • batch size-free	• G 수에 민감	Object Detection, Segmentation

Regularization — Dropout

Training: 일부 neuron을 random하게 off (zero)

- Probability: hyper-parameter. **0.5(**흔한 예**)**.
- 매번 다른 neuron 조합으로 학습.

효과

- **co-adaptation(**특정 feature이나 neuron에 지나치게 의존) 방지.
- **Ensemble 관점**: 매우 많은 partial model을 공유 parameter로 학습하는 ensemble masking neuron이 바뀔 때마다 서로 다른 partial network 학습.

Test: 모든 뉴런 사용 + 크기 보정

- 그대로 사용하면 학습 때보다 다음 층에 전달되는 값이 커짐 → **학습 시 기대값에 맞춰 보정**.
- **p: neuron 유지할 확률**.

---

2. Activation Functions — nonlinearity

일반적으로 **FC layer · CN layer 같은 선형 연산 뒤**에 배치.

Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$

- 범위: [0, 1]
- **한계: 기울기 소실(vanishing gradient).**
  - 입력이 큰 양수 또는 큰 음수 → 함수의 평평한 영역 도달 (**포화**).
  - 여러 layer 통과 → 앞쪽 층까지 전달되는 gradient가 작아짐.

ReLU: $f(x) = \max(0, x)$

- **장점:** 양수 영역 ← 포화 X. 계산이 간단하고 효율적. Sigmoid보다 빠르게 수렴.
- **한계:** 음수 영역 ← 출력, 기울기 = 0. 학습이 진행되지 않는 **Dead ReLU** 발생 가능.

GELU: $f(x) = x\Phi(x)$. (Φ: **표준정규분포의 누적분포함수.)**

- 장점: 0 주변에서 부드럽게 변화 (미분 가능) → 실제 학습에 도움을 줌.
- 단점: ReLU보다 계산 비용이 큼. 매우 큰 음수에서의 Gradient 문제 해결 X.

기타

Leaky ReLU: $f(x) = \max(0.1x, x)$ → 음수 영역에도 작은 기울기를 둠.

ELU: $f(x) = \begin{cases} x & (x \geq 0) \\ \alpha(e^x - 1) & (x < 0) \end{cases}$ → 음수 영역을 지수함수로 연결.

SiLU: $f(x) = x\sigma(x)$ → 입력에 Sigmoid 값을 곱함.

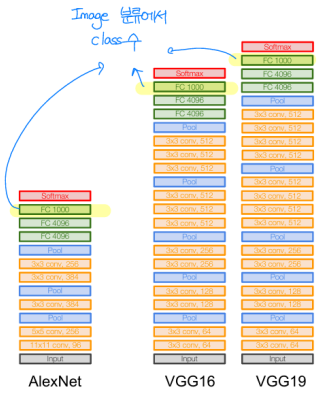
---

3. CNN Architectures (Case Study)

ImageNet 대회에서 나타난 흐름: model의 depth 증가 + 인식 성능 향상.

VGGNet

작은 filter 여러 개→ deeper



- AlexNet 8층 → VGG 16~19층.
- **작은 size Conv 여러 번 + Pooling** 연산

  ⇒ 같은 receptive field의 다른 구성. (기존: 더 큰 Conv layer.)

  ⇒ 더 많은 non-linearity 부여 가능. 학습 parameter 수 감소.
- Channel 수: **64** → **128** → **256** → **512** 증가.
- 마지막 layer: FC 층 + 분류 출력.

ResNet — Residual Block

deeper layer의 성능 향상

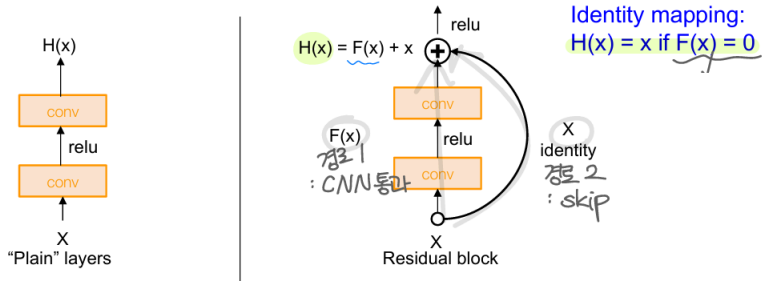
문제(Degradation Problem): 56층 모델이 20층 모델보다 test & training 오차 높음.

⇒ 최적화의 어려움. (추가된 layer가 정말 유용한가?)

Idea: Residual Block ⇒ 잔차를 학습

- 기존 층 H(x): 얕은 모델에서 학습한 변환 유지.
- 추가된 층: 입력을 그대로 통과시키는 **항등 매핑(identity mapping)** 수행.

  H(x) = x + F(x). (**F(x) = ReLU(x → 추가된 layer 통과)**)
  - Conv 경로: **입력에 추가할 변화량(residual) F(x)** 학습.
  - 추가 변환이 필요하지 않은 경우, **residual =** 0.



성과

- ImageNet용 **152층 모델**.
- ILSVRC 2015 분류 우승, **Top-5 오류율 3.57%**.
- ILSVRC 2015·COCO 2015의 분류·검출 대회에서도 우수한 성과.

---

4. Weight Initialization

초기 weight 크기: layer을 통과하는 activation 분포에 영향을 줌.

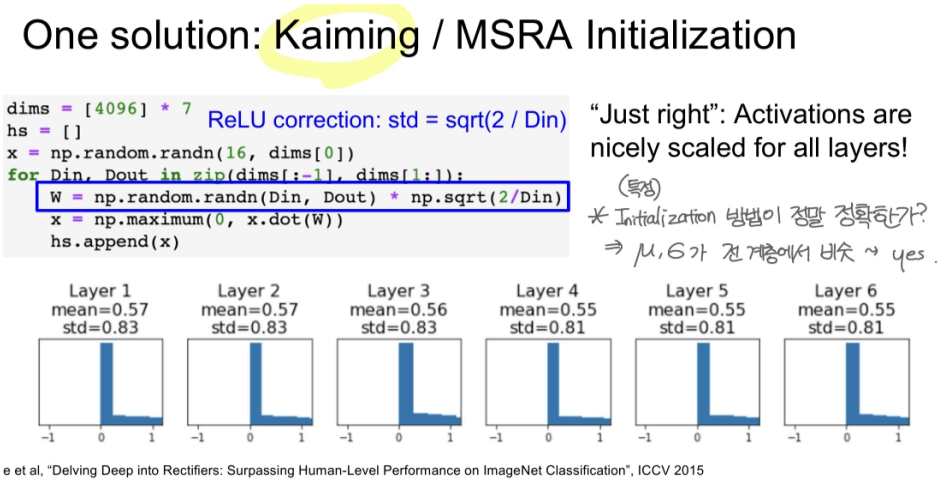
- 너무 작을 때: 뒤쪽 층으로 갈수록 활성값 분포가 **0 부근** ⇒ 신호 유지 X.
- 너무 클 때: 층을 지날수록 빠르게 커짐 ⇒ **활성값 폭발**.

⇒ Initialization 기준: **깊어져도 활성값 분포가 급격히 축소되거나 폭발하지 않도록 설정**.

(예시) Kaiming / MSRA Initialization

ReLU를 사용하는 경우, 입력 차원에 맞춰 표준편차 설정

- 입력 차원이 클수록 초기 weight 작게 설정.
- $\text{std}(W) = \sqrt{\frac{2}{D_{\text{in}}}},\ W_{ij} \sim \mathcal{N}\left(0, \frac{2}{D_{\text{in}}}\right)$



How to Train

5. Data Preprocessing

Image Normalization

pixel 위치마다 별도의 평균을 사용하는 방식과 구분.

- 통계(μ, σ): dataset-dependent
- ImageNet에서 미리 계산된 통계를 사용하는 경우도 있음.

---

6. Data Augmentation

Regularization 주요 패턴

- **Training:** randomness z 추가.
  $y = f_W(x, z)$
- **Test:** 그 randomness의 평균 정도의 효과 (가끔 근사).
  $f(x) = \mathbb{E}_z[f_W(x, z)]$

Flow: image + 정답 label 읽기 → image **transform** → CNN → Loss 계산

- 입력 image 형태를 다양하게 바꾸며 학습.
- 목표: transform 이후에도 정답 유지.

Transform 예시

- **Horizontal Flip — 좌우 반전**
- **Random Crops and Scales — 무작위 자르기·크기 변경**
  정해진 여러 변환의 예측 평균. ⇒ 다른 크기·위치의 대상 학습
- **Color Jitter — 색상 관련 변화**
- **Cutout — 이미지 일부 가리기**

---

7. Transfer Learning(전이학습)

data 양 부족 해결: 큰 dataset에서 학습한 CNN → 새 dataset에 재사용.

⇒ 사전학습된 CNN의 표현을 다른 인식 문제에도 활용할 수 있음.

Application

① **Pretrain(Imagenet):** 기존 네트워크 학습. (**1000개 클래스** 분류.)

② **- 1. 작은 dataset (Class C개):**

- 기존 feature extraction ← **동결(freeze)**.
- 마지막 분류 layer ← **C개 출력**으로 교체. re-initialize + train.

② **- 2. 큰 dataset**

- layer을 더 train하는 게 더 나음. Pretrained model에서 initialize.
- 더 많은 층 or 전체 모델 → **fine-tuning**

⇒ 데이터가 많아질수록 **더 많은 층을 새 데이터에 맞춰 학습**할 수 있음.

Data 개수 x (Pretrained model과의)Data 유사성

판단 기준: 데이터 수 + 사전학습 데이터와의 유사성.

새 데이터 양 \ 유사도	매우 유사	매우 다름
매우 적음	final layer의 linear classifier 사용	다른 모델을 시도 or data 추가 수집
충분함	모든 model layer의 fine-tuning	full fine-tuning or 전체 재학습

활용 방식

관심 dataset이 **약 100만 장 미만** → 유사한 large dataset에서 학습한 model을 찾은 뒤, 전이학습 고려.

---

8. Hyperparameter Selection

절차

1. initial Loss 확인: 학습 시작 시점의 Loss가 예상에 맞는지 확인.

2. 작은 sample에 overfitting

- small data에 충분히 맞출 수 있는지 확인.
- 이후 탐색은 이 단계에서 사용한 구조를 기준으로 진행.

3. Loss 줄이는 Learning Rate 찾기

- 전체 학습 데이터 사용. + 작은 **weight decay** 적용.
- 약 100번의 반복 안에 Loss가 뚜렷하게 감소하는 학습률 탐색.
- (예시) $10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}$

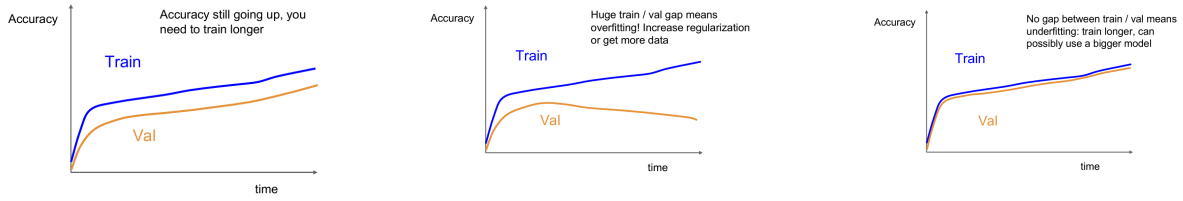
4. 넓은 범위로 탐색: 여러 hyperparameter 조합을 **약 1~5 epoch** 학습.

5. 범위를 좁히고 더 오래 학습: 좋은 결과가 나온 구간 탐색.

6. Loss·Accuracy 곡선 확인: 학습 상태를 진단 → 다음 조정 결정.

7. 5단계로 돌아가 반복

Loss·Accuracy 곡선



곡선의 형태	해석	대응
둘 다 적당히 상승	개선 중	더 오래 training
training 상승 + val 하강	overfitting	regularization 강화 or data 추가
(낮은 수준) 별 차이 X	underfitting	더 오래 학습 or 더 큰 모델 고려

Random Search vs. Grid Search

Grid Search: 정해진 격자의 모든 조합을 평가.

- 중요하지 않은 parameter 방향에도 같은 수의 실험 배분.
- 일부 parameter만 성능에 크게 영향 ⇒ 실험을 비효율적으로 사용.

Random Search: 조합을 무작위로 선택.

- 같은 실험 횟수에서도 **중요한 parameter의 더 다양한 값**을 평가할 수 있음.
- ⇒ parameter별 중요도가 다를 때, **탐색 예산을 더 효과적으로 사용할 수 있음**.

강의 정리

1. **CNN 구성 층:** 합성곱·FC·정규화·Dropout의 역할 이해.
2. **활성화 함수:** 비선형성 추가, 포화와 기울기 특성 확인.

3. **CNN 구조:** VGG의 작은 필터 적층, ResNet의 잔차 연결.
4. **가중치 초기화:** 층을 지나도 활성값 분포가 유지되도록 설정.
5. **데이터 전처리:** 채널별 평균을 빼고 표준편차로 나누기.
6. **데이터 증강:** 자르기·크기 변경·밝기와 대비 변화 등으로 입력 다양화.
7. **전이학습:** 사전학습 표현을 데이터 양·유사성에 맞춰 재사용.
8. **하이퍼파라미터 선택:** 초기 손실·소량 과적합 확인 → 학습률 탐색 → 곡선 진단과 반복 탐색.